Unit-1

Introduction of Data Structure

Unit-1

- Introduction of Data
- Introduction of Data type
- Introduction of Data Structure
- Primitive and non-primitive data structure
- Complexity of Data Structure
- Analysis of Algorithm

Introduction of Data

- **Data** is distinct pieces of information.
- Data can exist in a variety of forms:
  - Numbers or texts on pieces of paper
  - Bits or bytes stored in electronic memory
  - Facts stored in human's mind
- People have used the word **DATA** to mean computer information that is transmitted or stored.
- Strictly speaking data is a **plural of datum**, a single piece of information.
- In practice, however, people use data as both the singular and plural form of the word.

Introduction of Datatype

- **Data types** specify how we enter data into our programs and what type of data we enter.
- C language has some predefined set of data types to handle various kinds of data that we can use in our program.
- C data types are used to:

- Identify the **type of the variable** when it is declared.
- Identify the **type of the return value of a function**.
- Identify the **type of the parameter expected by a function**.

Introduction of Datatype

- C language supports three different data types:
  1. **Primary data types**
  2. **Derived data types**
  3. **User defined data types**

Primary (Built-in) Data types:

void, int, char, float and double

Derived Data types:

Arrays and Pointers

User Defined Data types:

Structure, Union and Enumeration

Primary Datatype

Table : Primary data types in C

Data Type	Keyword used	Size in bytes	Range	Use
Character	char	1	-128 to 127	To store characters
Integer	int	2	-32768 to 32767	To store integer numbers
Floating point	float	4	3.4E -38 to 3.4E +38	To store floating point numbers
Double	double	8	1.7E -308 to 1.7E +308	To store big floating point numbers
Valueless	void	0	Valueless	-----

Derived Data type

- **Array**

- An array is a collection of data items, all of the same type, accessed using a common name. A one-dimensional array is like a list, A two dimensional array is like a table.

Derived Data type

```
#include<stdio.h>
#include<conio.h>

void main()
{
	int a[50],i,n;
	clrscr();
	printf("\n Enter Limit : ");
	scanf("%d",&n);
	for(i=0;i<n;i++)
	{
		printf("\n Enter Value %d : ",i+1);
		scanf("%d",&a[i]);
	}
	printf("\n Entered Values are as follows \n");
	for(i=0;i<n;i++)
	{
		printf("\n%d",a[i]);
	}
	getch();
}
```

Derived Data type

- **Pointer**

- A **pointer** is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a **pointer** before using it to store any variable address.

Derived Data type

```
#include<stdio.h>
#include<conio.h>

void swap(int *,int *);

void main()
{
	int a=10,b=20;
	clrscr();
	printf("\n Before Swap A : %d and b : %d",a,b);
	swap(&a,&b);
	printf("\n After Swap A : %d and b : %d",a,b);
	getch();
}
void swap(int *a,int *b)
{
	int temp;
	temp=*a;
	*a=*b;
	*b=temp;
}
```

User Defined Datatype

- **Structure**
- Structure is a user-defined data type in C language which allows us to combine data of different types together. Structure helps to construct a complex data type which is more meaningful. It is somewhat similar to an Array, but an array holds data of similar type only.

```
#include<stdio.h>
#include<conio.h>

struct student
{
    int rollno;
    char name[20];

};
```

User Defined Datatype

```
void main()
{
      struct student s1;
      clrscr();

      printf("\n Enter Roll Number : ");
      scanf("%d",&s1.rollno);

      printf("\n Enter Name : ");
      fflush(stdin);
      gets(s1.name);

      printf("\n Student Data is as follows ");
      printf("\n **************************");

      printf("\n Roll Number is : %d",s1.rollno);
      printf("\n Name is : %s",s1.name);

      getch();
}
```

User Defined Datatype

- **Union**

- A union is a special data type available in C that allows to store different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time.

```
#include<stdio.h>
#include<conio.h>

union student
{
    int rollno;
};
```

User Defined Datatype

```
void main()
{
    union student s1;
    clrscr();

    printf("\n Enter Roll Number : ");
    scanf("%d",&s1.rollno);

    printf("\n Student Data is as follows ");
    printf("\n **************************");

    printf("\n Roll Number is : %d",s1.rollno);
    getch();
}
```

User Defined Datatype

- **Enum**

- Enumeration is a user defined datatype in C language. It is used to assign names to the integral constants which makes a program easy to read and maintain. The keyword “enum” is used to declare an enumeration.

User Defined Datatype

```
#include<stdio.h>
#include<conio.h>

enum week{Mon,Tue,Wed,Thu,Fri,Sat,Sun};

void main()
{
    enum week w;
    clrscr();
    w=Tue;

    printf("\n Constant Value of Tue is : %d",w);
    getch();
}
```

Introduction of Data Structure

- **Data structure** is a way of collecting and organizing data in such a way that we can perform operations on these data in an effective way.

- Data structure is about rendering (providing) data elements in terms of some relationship, for better organization and storage.

Introduction of Data Structure

- **For example,** we have data player's name "Hitesh" and age 26.
- Here we have two data elements:
  - Player's name: string datatype
  - Player's age: integer datatype
- We can organize this data as a record like Player record.
- Now we can collect and store Player's records in a file or database as a data structure.
- For example, "Gayle" 30, "Sachin" 31, "Parth" 33.
- In simple language, Data structures are structures programmed to store ordered data, so that various operations can be performed easily.

Introduction of Data Structure

- Anything that can store data can be called as a data structure, hence Integer, Float, Boolean, Char etc… are also called data structures. They are also known as **Primitive Data Structures/ In-Built Data Structures**.

- Complex type of data structures are also available, which are used to store large and connected data. They are also known as **Non-Primitive Data Structures/ Derived Data Structures/ Abstract Data Structures.** For example, Array, Stack, Queue, Linked list, Tree, Graph etc….

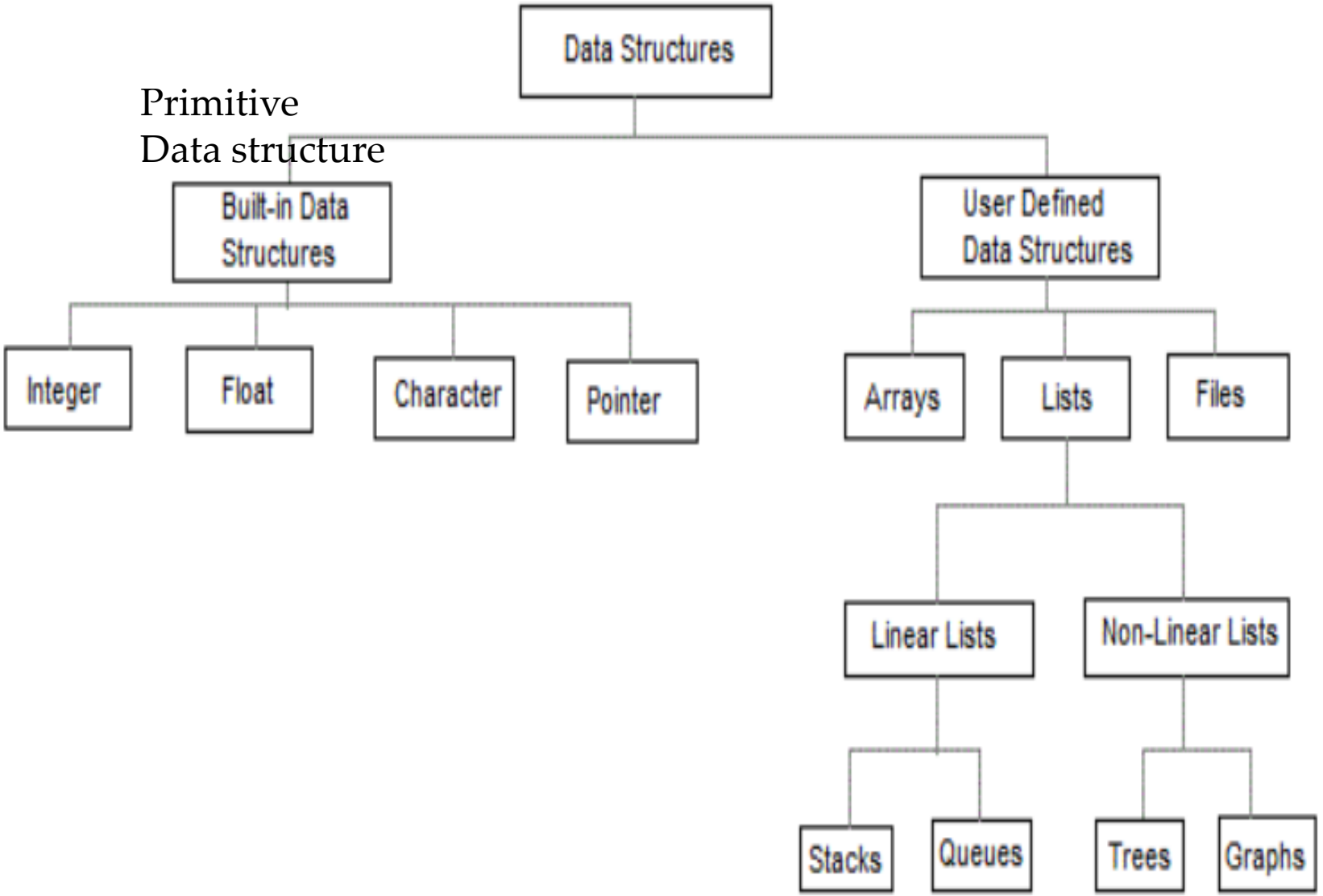


Figure : Classification of Data Structures

Introduction of Data Structure

- The data structures can also be classified on the basis of the following characteristics:

- **Linear :**
  - In Linear data structures, the data items are arranged in a linear sequence.
  - Example: Array,Stack,Queue

- **Non-Linear:**
  - In Non-Linear data structures data items are not in sequence.
  - Example: Tree, Graph

Introduction of Data Structure

❖ **Homogeneous :**

- In Homogeneous data structures, the elements are of same type.
- Example: Array

❖ **Non- Homogeneous :**

- In Non- Homogeneous data structures all elements may or may not be of same type.
- Example: Structures

Introduction of Data Structure

❖ **Static :**

- Static data structures are those whose sizes and structures associated memory locations are fixed at compile time.
- Example: Array

❖ **Dynamic :**

- Dynamic data structures are those which expands and shrinks depending upon the program need and its execution.
- Also their associated memory locations change.
- Example: Linked lists created using pointers

Introduction of Data Structure

- The data in the data structures are processed by certain operations.
- The particular data structure chosen, largely depends on the frequency of the operation that needs to be performed on the data structure.

  - **Traversing**
  - **Searching**
  - **Insertion**
  - **Deletion**
  - **Sorting**
  - **Merging**

Complexity of Data Structure

Complexity is a rough approximation of the number of steps necessary to execute an algorithm.

Two types of Complexity of Data Structure

1. Time Complexity
2. Space Complexity

Complexity of Data Structure

1. Time Complexity

The time complexity of an algorithm is basically the running time of a program as a function of the input size.

```
for(i=0; i < N; i++)
{
   statement;
}
```

The time complexity for the above algorithm will be Linear. The running time of the loop is directly proportional to N. When N doubles, so does the running time.

Complexity of Data Structure

2. Space Complexity

- Space complexity of an algorithm represents the amount of memory space needed the algorithm in its life cycle.
- Space needed by an algorithm is equal to the sum of the following two components
- A fixed part that is a space required to store certain data and variables (i.e. simple variables and constants, program size etc.), that are not dependent of the size of the problem.
- A variable part is a space required by variables, whose size is totally dependent on the size of the problem. For example, recursion stack space, dynamic memory allocation etc.

Analysis of Algorithm

We can analyze algorithm in Three Different ways as follows.

1. Best Case
2. Worst Case
3. Average Case

Analysis of Algorithm

1. Best Case

The best-case complexity of the algorithm is the function defined by the minimum number of steps taken on any instance of size n.

The term 'best-case performance' is used to analyse an algorithm under optimal conditions.

Analysis of Algorithm

2. Worst Case

The worst-case complexity of the algorithm is the function defined by the maximum number of steps taken on any instance of size n.

The worst-case running time of an algorithm is an upper bound on the running time for any input.

Therefore, having the knowledge of worst-case running time gives us an assurance that the algorithm will never go beyond this time limit.

Analysis of Algorithm

3. Average Case

The average-case running time of an algorithm is an estimate of the running time for an 'average' input.

It specifies the expected behaviour of the algorithm when the input is randomly drawn from a given distribution.

Thank You…!!!!